Binary Search Tree (BST) — C Program

Operations: insert, longest root-to-leaf path, minimum value, mirror, search, and inorder traversal.

Program Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int value) {
    if (root == NULL) return createNode(value);
    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    return root;
}

int height(struct Node* root) {
    if (root == NULL) return 0;
    int leftHeight = height(root->left);
    int rightHeight = height(root->right);
    return 1 + (leftHeight > rightHeight ? leftHeight : rightHeight);
}

int minimum(struct Node* root) {
    if (root == NULL) return 999999;
    int min = root->data;
    int leftMin = minimum(root->left);
    int rightMin = minimum(root->right);
    if (leftMin < min) min = leftMin;
    if (rightMin < min) min = rightMin;
    return min;
}

struct Node* search(struct Node* root, int key) {
    if (root == NULL || root->data == key) return root;
    struct Node* result = search(root->left, key);
    if (result != NULL) return result;
    return search(root->right, key);
}

void mirror(struct Node* root) {
    if (root == NULL) return;
    struct Node* temp = root->left;
    root->left = root->right;
```

```
    root->right = temp;
    mirror(root->left);
    mirror(root->right);
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    struct Node* root = NULL;
    int n, value, choice, key, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter values: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    do {
        printf("\\n\\n--- BST OPERATIONS ---\\n");
        printf("1. Insert new node\\n");
        printf("2. Longest path from root\\n");
        printf("3. Minimum data value\\n");
        printf("4. Mirror the tree\\n");
        printf("5. Search a value\\n");
        printf("6. Display inorder\\n");
        printf("0. Exit\\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                root = insert(root, value);
                printf("Node inserted.\\n");
                break;
            case 2:
                printf("Nodes in longest path: %d\\n", height(root));
                break;
            case 3:
                if (root == NULL) printf("Tree is empty.\\n");
                else printf("Minimum value: %d\\n", minimum(root));
                break;
            case 4:
                mirror(root);
                printf("Tree mirrored successfully.\\n");
                break;
            case 5:
                printf("Enter value to search: ");
                scanf("%d", &key);
                if (search(root, key) != NULL)
                    printf("%d found in the tree.\\n", key);
                else
                    printf("%d not found in the tree.\\n", key);
```

```
                break;
            case 6:
                printf("Inorder traversal: ");
                inorder(root);
                printf("\\n");
                break;
            case 0:
                printf("Exiting program.\\n");
                break;
            default:
                printf("Invalid choice.\\n");
        }
    } while (choice != 0);
    return 0;
}
```

Sample Output

Initial values: 50, 30, 70, 20, 40, 60, 80. Then insert 35 and run the operations shown.

```
Enter number of nodes: 7
Enter values: 50 30 70 20 40 60 80

--- BST OPERATIONS ---
Enter choice: 6
Inorder traversal: 20 30 40 50 60 70 80

--- BST OPERATIONS ---
Enter choice: 1
Enter value to insert: 35
Node inserted.

--- BST OPERATIONS ---
Enter choice: 2
Nodes in longest path: 4

--- BST OPERATIONS ---
Enter choice: 3
Minimum value: 20

--- BST OPERATIONS ---
Enter choice: 5
Enter value to search: 60
60 found in the tree.

--- BST OPERATIONS ---
Enter choice: 4
Tree mirrored successfully.

--- BST OPERATIONS ---
Enter choice: 6
Inorder traversal: 80 70 60 50 40 35 30 20

--- BST OPERATIONS ---
Enter choice: 3
Minimum value: 20

--- BST OPERATIONS ---
Enter choice: 5
Enter value to search: 40
40 found in the tree.

--- BST OPERATIONS ---
Enter choice: 0
```

```
Exiting program.
```

Note: After inserting 35, the longest path contains 4 nodes. The minimum value remains 20 after mirroring.